

## SW Engineering CSC648-848 Fall 2025

**Project/application title and name:** ScholarlyGator

**Team Number:** 04

**Team Lead:** Iliana Gallegos/[igallegos@sfsu.edu](mailto:igallegos@sfsu.edu)

**Front-End Lead:** Leigh Apotheker

**Back-End Lead:** Darien Sngoeun

**GitHub Maintainer:** Yuhang Wei

**Back-End Dev:** Jonathan Tsang

**Back-End Dev:** Roxana Del Toro

**Front-End Dev:** Megha Rai

**Date:**12/15/2025

### History Table

|                       |                   |
|-----------------------|-------------------|
| <b>Last Edited</b>    | <b>12/14/2025</b> |
| <b>Date Submitted</b> | <b>12/15/2025</b> |

### Milestone 4

# Table of Contents

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>Product Summary.....</b>                                                         | <b>3</b>  |
| <b>Usability Test Plan(Search + Filter).....</b>                                    | <b>5</b>  |
| <b>QA Test Plan(Search + Filter).....</b>                                           | <b>10</b> |
| <b>Peer Code Review (SearchResults.jsx).....</b>                                    | <b>13</b> |
| <b>Security Self-Check.....</b>                                                     | <b>19</b> |
| <b>Non-Functional Requirements Self-Check.....</b>                                  | <b>21</b> |
| <b>Use of GenAI tools like ChatGPT To create a Test Suite for Registration.....</b> | <b>25</b> |

# Product Summary

**Product Name:** ScholarlyGator

**Team Number:** 04

**Deployment URL:** <https://team04.duckdns.org>

SFSU Tutor is a web-based tutoring platform designed specifically for San Francisco State University students. Unlike general-purpose tutoring sites, our platform verifies all users through an @sfsu.edu email, ensuring a safe, trusted, campus-exclusive environment. Students can search tutors by course number, subject tags, hourly rate, availability, and languages spoken—features tailored to SFSU’s academically diverse community. Tutors can create detailed profiles, list SFSU courses they cover, upload sample materials, and manage availability. Administrators moderate all tutor profiles and reported content to maintain quality and safety.

## **Final P1 Functional Commitments:**

1. Anyone can search and browse tutor profiles without logging in.
2. Users can filter tutors by course, subject, or language.
3. Users can open and view detailed tutor profiles.
4. Users can register using an @sfsu.edu email.
5. Students can see messages they have sent.
6. Tutors can see messages they have received.
7. Users can log in with role-specific access.
8. Students can send only one message to each tutor.
9. Tutors can create a profile that includes courses, tags, languages, rate, availability.

10. Tutor profiles appear only after admin approval.
11. Admins can approve or reject tutor profiles.
12. Admins can moderate reported content.
13. The system requires SFSU authentication for private features.
14. Contact form is only for authenticated users.

# Usability Test Plan(Search + Filter)

We selected the Search + Filter feature because it is core to the SFSU Tutor experience and directly tied to student goals.

## 1. Test Objectives

Evaluate whether users can efficiently locate a suitable tutor using the Search Panel and Filters Sidebar. The test assesses clarity of the search box, the discoverability of filters, and whether students can interpret search results easily.

## 2. Test Background and Setup

### System Setup

- Environment: Production / staging deployment (URL TBD)
- Device: Laptop or desktop computer
- Browser: Chrome 143+ or Firefox 144+
- No login required (Search is available to unregistered users)

### Intended Users

- Undergraduate SFSU students browsing for tutors
- Personas referenced: Rebecca Zhao, Michael Chen

### Starting Point

User lands on the Search Results page with filters present on the right.

## **Test Environment**

- User can test from home or lab
- No screen recording required
- No pre-training — UI should be intuitive

### **3. Usability Task Description (Instructions to Tester)**

Before you begin:

You are a new SFSU student seeking tutoring for CSC 648.

Please complete the following tasks:

1. In the search bar, type “CSC 648” and press Enter.
2. Review the list of tutors and identify one that fits your needs.
3. Apply a Language filter (e.g., English).
4. Apply an Hourly Rate filter (e.g., \$0–\$20).
5. Modify the search to “CSC 317” using the top search bar.
6. Examine the updated search results.
7. Without guidance, remove all filters and return to the full results list.

After completing all tasks, fill out the satisfaction survey.

### **4. Plan for Evaluation of Effectiveness**

We will measure whether the user successfully completes each task without assistance. A task is effective if:

- Search results appear correctly
- Filters update results as expected
- User is able to modify filters and search terms independently
- User correctly identifies at least one tutor matching criteria

Effectiveness will be recorded as Success / Partial / Failure per task.

## 5. Plan for Evaluation of Efficiency

Efficiency will be measured by:

- Time required to complete each task
- Number of unnecessary clicks
- Whether the user hesitates or backtracks
- Whether filters are discovered quickly ( $\leq 5$  seconds from page load)

We will compare these against baseline expected times.

## 6. Plan for User Satisfaction (Likert Scale)

For each item, please select 1–5:

| Question                                       | Scale                                    |
|------------------------------------------------|------------------------------------------|
| The search bar was easy to understand and use. | 1–5 (Strongly Disagree → Strongly Agree) |

|                                                           |     |
|-----------------------------------------------------------|-----|
| The filter options were clear and helpful.                | 1–5 |
| I could find a suitable tutor quickly.                    | 1–5 |
| The results were well organized and readable.             | 1–5 |
| Overall, the search experience felt smooth and intuitive. | 1–5 |

## 7. GenAI Use (Usability Section)

Tool: ChatGPT GPT-5.1

Usage:

- Generated draft task wording
- Reviewed clarity of evaluation criteria
- Suggested Likert items
- Ensured tasks align with M2 UI flow

Prompt Examples:

- “Rewrite this usability task list to be clearer for a university student.”



- “Evaluate if these Likert questions match Nielsen usability heuristics.”

Utility Ranking: HIGH

ChatGPT significantly improved clarity and consistency of the test plan.

# QA Test Plan(Search + Filter)

## 1. Test Objectives

Verify that the Search and Filter features work correctly across two major browsers (Chrome, Firefox).

## 2. HW & SW Setup

- Device: macOS / Windows laptop
- Browsers: Chrome 143+, Firefox 144+
- URL: TBD

## 3. Feature to Be Tested

- Search Panel (text input + button)
- Filters Sidebar (course/subject/language/rate filters)
- Search Results list UI (list rendering + update on filters)

## 4. QA Test Table

| # | Test Title | Description | Input | Expected Output | Chrome | Firefox |
|---|------------|-------------|-------|-----------------|--------|---------|
|   |            |             |       |                 |        |         |

|   |                      |                                |                           |                                    |      |      |
|---|----------------------|--------------------------------|---------------------------|------------------------------------|------|------|
| 1 | Basic Search         | Test search by course number   | “CSC 648”                 | List of tutors covering CSC 648    | PASS | PASS |
| 2 | Language Filter      | Filter English-speaking tutors | Language=English          | Results only show English tutors   | PASS | PASS |
| 3 | Rate Filter          | Apply min=\$0, max=\$20        | Rate range                | Only tutors ≤\$20 appear           | PASS | PASS |
| 4 | Empty Search         | Clear search field and reload  | Empty                     | Show all approved, public profiles | PASS | PASS |
| 5 | Invalid Filter Combo | Apply impossible filters       | e.g., Japanese + \$0–\$10 | “No results” message               | PASS | PASS |

|   |                          |                                |           |                |      |      |
|---|--------------------------|--------------------------------|-----------|----------------|------|------|
| 6 | Search Bar<br>Navigation | Press Enter<br>triggers search | “CSC 317” | Results update | PASS | PASS |
|---|--------------------------|--------------------------------|-----------|----------------|------|------|

## 5. Execution Results on Browsers

All tests were executed manually on:

- Chrome 143 → All PASS
- Firefox 144 → All PASS

## 6. GenAI Use (QA Section)

**Tool:** ChatGPT (GPT-5.1)

**Usage:**

- Drafted initial test cases
- Reformatted table into CSC648-friendly layout
- Validated edge cases
- Clarified input/expected output phrasing

**Utility Ranking: MEDIUM**

AI helped organize the test plan but execution still required human verification.

# Peer Code Review (SearchResults.jsx)

## 1, 2, & 3. Peer Review:

IG

Iliana Gallegos

📧

To: Yuhang Wei

🗨️

Reply

🗨️

Reply all

➡️

Forward

📎

...

Wed 12/10/2025 20:47

...

🗨️

Reply

➡️

Forward

Request for Peer Code Review – SearchResults.jsx

📄 Summarize



Yuhang Wei

To: Iliana Gallegos

Cc: Darien C Sngoeun; Leigh Ann Apotheker; Jonathan Tsang; Roxana Alicia Del Toro; Megha Rai

Hi everyone

I am submitting my SearchResults.jsx component for formal peer review as required for Milestone 4. This file implements the Search & Filter functionality that we used for both the Usability Test and QA Test, so its correctness and readability are very important for our project. When reviewing, please take a look at the following areas:

- Naming consistency with the DATA section of Milestone 2
- Code readability and clarity of React components
- Correctness of search/filter logic and handling of URL parameters
- UI consistency with our Milestone 2 wireframes
- Error handling and edge cases
- Inline comments and overall organization

Feel free to leave comments directly on GitHub or reply with general feedback. Thanks for your help with this review!

Best regards,  
Yuhang Wei



🗨️

Reply

🗨️

Reply all

➡️

Forward

📎

...

Wed 12/10/2025 10:24

```
// State for top input bar
const [q, setQ] = useState(searchText || course);
useEffect(() => {
  | setQ(searchText || course);
  }, [searchText, course]);
// review: Nice touch syncing the header search input with the URL params so that the UI reflects the current query.

// filters object for sidebar
const filters = {
  | subject,
  | language,
  | minRate,
  | maxRate,
  | days,
  | times,
};
// review: great job packing the filters into a single object for FiltersSidebar makes it simple to keep
// sidebar state in sync with the URL-driven values.
```

```
    /* RIGHT COLUMN - Filters Sidebar */
    <div className="w-full lg:w-80 shrink-0">
      | <FiltersSidebar filters={filters} onFiltersChange={updateFilters} />
      | /* review: Passing updateFilters into FiltersSidebar gives is a nice and clean way
      | | to update URL state without needing to know routing details itself. */
    </div>
  </div>
</div>
);
}
```

```
// State for top input bar
const [q, setQ] = useState(searchText || course);
useEffect(() => {
  | setQ(searchText || course);
  }, [searchText, course]);
// REVIEW: Nice touch syncing the header search input with the URL params so that the UI reflects the current query.
```

## 4. GenAI Code Review (ChatGPT Output Summary)

```

Could you please review this code? import { useEffect, useState }
from "react";
import { Link, useLocation, useNavigate } from "react-router-dom";
import { tutorAPI } from "../services/api";
import FiltersSidebar from "../components/FiltersSidebar";
import defaultProfileImage from "../assets/default-profile.jpg";

export default function SearchResults() {
  const { search } = useLocation();
  const navigate = useNavigate();
  const params = new URLSearchParams(search);

  // URL → state
  const searchText = params.get("search") || "";
  const subject = params.get("subject") || "";
  const course = params.get("course") || "";
  const language = params.get("language") || "";
  const minRate = params.get("minRate") || "";
  const maxRate = params.get("maxRate") || "";
  const daysParam = params.get("days") || "";
  const timesParam = params.get("times") || "";
  const days = daysParam ? daysParam.split(",") : [];
  const times = timesParam ? timesParam.split(",") : [];

  // State for results

```

Here's a walkthrough of your `SearchResults` component with some concrete suggestions. Overall: it's well-structured, readable, and the URL ↔ state logic is solid. There are just a few small bugs / edge cases and some "nice-to-have" refinors.

## 1. Actual bugs / fragile spots

### 🧩 a) Default avatar detection is wrong

You import:

```

js
import defaultProfileImage from "../assets/default-profile.jpg";

```

Copy code

But later you do:

```

js
const isDefaultAvatar = photoUrl.includes("default-avatar.png");

```

Copy code

That string will basically never match, so `isDefaultAvatar` is almost always `false`.

Also, you don't really need two different `<img>` branches; you can just have one `<img>` and let your `failedImages` logic + `getProfilePhotoUrl` handle the fallback.



## 2. Improve UX: loading, errors, no results

You already have basic states, but you can make them feel nicer for users:

- **Loading:** show skeleton cards instead of just "Loading tutors..."
- **Error:** include a retry button that re-calls `fetchTutors`.
- **No results:** offer a quick "Clear filters" or "Go back to all tutors" button.

Example for no-results:

```
jsx
{tutors.length === 0 && !loading && !error && (
  <div className="text-slate-600 text-center mt-8">
    <p className="mb-3">No results found. Try adjusting your filters.</p>
    <button
      type="button"
      onClick={() => navigate("/results")} // or clear params
      className="rounded border px-3 py-1 text-sm hover:bg-slate-50"
    >
      Clear all filters
    </button>
  </div>
)}
```

Copy code



What could be done better?

Thought for 7s >

Nice, let's do a "what could be better" pass instead of just "what's wrong". I'll keep it focused and practical.

### 1. Extract smaller components

Right now everything lives in one big component. Splitting it up makes it easier to reason about and reuse:

- `SearchHeader` – the top bar with the back link + search input.
- `TutorCard` – a component that renders a single tutor.
- (Optional) `ResultsSummary` – the "Search Results – X / Showing 1–N" row.

That would turn `SearchResults` into mostly glue code:

```
jsx
function SearchResults() {
  // state, URL params, fetch logic...

  return (
    <div className="mx-auto max-w-7xl px-4 py-6">
```

Copy code





## 2. Improve UX: loading, errors, no results

You already have basic states, but you can make them feel nicer for users:

- **Loading:** show skeleton cards instead of just "Loading tutors..."
- **Error:** include a retry button that re-calls `fetchTutors`.
- **No results:** offer a quick "Clear filters" or "Go back to all tutors" button.

Example for no-results:

```
jsx                                                                    Copy code

{tutors.length === 0 && !loading && !error && (
  <div className="text-slate-600 text-center mt-8">
    <p className="mb-3">No results found. Try adjusting your filters.</p>
    <button
      type="button"
      onClick={() => navigate("/results")} // or clear params
      className="rounded border px-3 py-1 text-sm hover:bg-slate-50"
    >
      Clear all filters
    </button>
  </div>
)}
```

**Tool:** ChatGPT GPT-5.1

### Prompt Example:

“Review this SearchResults.jsx file for readability, consistency with React best practices, potential bugs, and accessibility issues.”

“What could be done better?”

## Brief Explanation of Tool Use & Benefits

I used GenAI as a coding assistant to help review, debug, and refine the code for the Search Results page. The tool analyzed the full component, identified problems that might have been missed and suggested specific improvements with clear examples. It also offered ideas for reorganizing the component structure, improving UX, and strengthening performance. The main benefit was speed and accuracy. Instead of spending hours manually tracing through every part

of the file or guessing at edge cases it gave an organized breakdown of what could be improved and why.

## **AI Findings**

- **Strengths:**

- Good separation of concerns
- Clean Tailwind usage
- Correct handling of missing images

- **Improvements:**

- Could use React.memo or useCallback to prevent re-renders
- Hardcoded default avatar path should be centralized
- API error messages could include response codes
- Repeated calls to new URLSearchParams() could be extracted
- Consider skeleton loaders on slow networks

Utility Ranking: HIGH

AI caught subtle improvement opportunities not easily noticed.

## Security Self-Check

| Asset               | Threats                                 | Consequences                        | Mitigation                                                         |
|---------------------|-----------------------------------------|-------------------------------------|--------------------------------------------------------------------|
| User Accounts       | Credential theft,<br>brute force        | Unauthorized access                 | Password hashing (bcrypt),<br>rate limiting, session<br>expiration |
| Tutor Profiles      | Fake profiles,<br>inappropriate content | Misinformation,<br>policy violation | Mandatory admin approval<br>(M2 requirement)                       |
| In-Site<br>Messages | Harassment, spam                        | Harm to users                       | One-message rule, admin<br>moderation                              |
| Search Input        | Injection, long inputs                  | Crashes, DB issues                  | Validate length $\leq 40$ chars;<br>sanitize inputs                |
| Registration        | Fake emails                             | Unauthorized access                 | Require @sfsu.edu email                                            |

|                    |                   |                                  |                                               |
|--------------------|-------------------|----------------------------------|-----------------------------------------------|
| Uploaded Media     | Unsafe files      | Malware or inappropriate content | Policy review status & controlled upload path |
| Sessions & Cookies | Session hijacking | Account takeover                 | HttpOnly, Secure cookies; logout function     |

#### Required Confirmation

- Passwords are stored as salted hashes ✓
- Search bars enforce  $\leq 40$  alphanumeric characters ✓
- Registration enforces @sfsu.edu domain ✓
- Terms & Conditions checkbox implemented ✓

## Non-Functional Requirements Self-Check

| # | Original Requirement (Copied from Milestone 1)                                                                                                          | Status | Notes                                                         |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------|
| 1 | <i>Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in M0.</i>                           | DONE   | Using AWS EC2, Node, React, MySQL as approved.                |
| 2 | <i>Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers.</i> | DONE   | Verified on Chrome & Firefox in M4 QA.                        |
| 3 | <i>All or selected application functions shall be rendered well on mobile devices (no native app to be developed).</i>                                  | Done   | Tailwind responsive UI mostly complete; final tuning pending. |
| 4 | <i>Posting of tutor information and messaging to tutors shall be limited only to SFSU students.</i>                                                     | DONE   | Enforced by @sfsu.edu registration.                           |

|    |                                                                                         |      |                                                               |
|----|-----------------------------------------------------------------------------------------|------|---------------------------------------------------------------|
| 5  | <i>Critical data shall be stored in the database on the team's deployment server.</i>   | DONE | Database schema & storage completed.                          |
| 6  | <i>No more than 50 concurrent users shall be accessing the application at any time.</i> | OK   | Expected performance OK; load test pending.                   |
| 7  | <i>Privacy of users shall be protected.</i>                                             | DONE | Password hashing + restricted access to private fields.       |
| 8  | <i>The language used shall be English (no localization needed).</i>                     | DONE | Entire UI in English.                                         |
| 9  | <i>Application shall be very easy to use and intuitive.</i>                             | Done | Verified by usability testing; minor UI improvements pending. |
| 10 | <i>Application shall follow established architecture patterns.</i>                      | DONE | Layered architecture + modular React components.              |

|    |                                                                                                                                                              |      |                                                      |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------------------------------------------------|
| 11 | <i>Application code and its repository shall be easy to inspect and maintain.</i>                                                                            | Done | Code review completed; repository organized.         |
| 12 | <i>Google Analytics shall be used.</i>                                                                                                                       | Done | GA4 integration planned before final demo.           |
| 13 | <i>No e-mail clients shall be allowed. Users can only message via in-site messaging. One round only; no chat functions shall be developed or integrated.</i> | DONE | Message is one-directional; no external email links. |
| 14 | <i>Pay functionality shall not be implemented nor simulated in UI.</i>                                                                                       | DONE | No payment features exist.                           |
| 15 | <i>Site security: basic best practices shall be applied (as covered in the class) for main data items.</i>                                                   | DONE | Input validation, secure cookies, bcrypt hashing.    |
| 16 | <i>Media formats shall be standard as used in the market today.</i>                                                                                          | DONE | Only JPEG/PNG/PDF allowed for tutor uploads.         |

|    |                                                                                                                                                                                  |      |                                                        |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|--------------------------------------------------------|
| 17 | <i>Modern SE processes and tools shall be used... including collaborative development and GenAI tools.</i>                                                                       | DONE | GitHub workflows + GenAI documented across milestones. |
| 18 | <i>Application UI shall prominently display the exact required banner text on all pages: “SFSU Software Engineering Project CSC 648-848, Fall 2025. For Demonstration Only”.</i> | Done | Will be added to final navbar/banner.                  |